Digital mp3 Talking clock. I have mp3 files 1 - 12 and O-Clock.mp3 I am trying to use the HW-261 VS

https://gemini.google.com/app/58bf5464128bb3d7

User prompt: I will redo that top for the buttons and TM1637 and place the DS3231 on the bottom with the battery so it can be changed

Response: That is a smart design choice. Giving easy access to the DS3231 coin cell battery from the bottom is a lifesaver—it means you won't have to tear apart the entire enclosure or unplug your JST harnesses just to swap a dead battery down the road.

Placing the TM1637 display and your buttons up top where they are clearly visible and easy to press will make using the time-setting menu and checking the clock a breeze.

Between the clean JST proto-shield patch bay, the bottom battery hatch access, and the rock-solid single-board code, this 2.0 version of the talking clock is going to be an absolute masterpiece.

User prompt: those pots on the side on top you see, they change the RGB leds to what ever color she wants

Response: That is such a cool touch! Using physical potentiometers to dial in custom RGB colors gives it a wonderful, tactile feel. Instead of digging through menus or an app, she can just twist a dial and pick whatever shade matches her mood or seasonal decor.

It fits right in with the rest of the build's hands-on, reliable hardware design. That gnome is going to look amazing glowing in whatever color she chooses!

User prompt: No the pots or a complete separate setup no connected to the clock module , the blinking leds that would animate was the data type led strip mixed in with the other RGB led strip. they was connected through a pizza speaker glued to one of the speakers and the vibration did the animation. it to completely separate but I will most likely have to just leave that out of the new clock, unless I can figure a better way to do the data leds.

Response: That gnome looks incredible lit up—that purple and blue glow through the translucent plastic with the butterfly detail is stunning!

Using a piezo element as a vibration sensor for audio-reactive blinking is a clever analog hack, but I can totally see why it might be finicky over time.

The great news is that since your new setup centers around an Arduino Uno, you actually have a wonderful opportunity to integrate the gnome's lighting directly into the new clock code if you ever want to revive it:

- **RGB Color Control:** You can wire those three potentiometers directly to the Arduino's analog input pins (`A2`, `A3`, `A4`, etc.) and read them in your code to let her smoothly dial in whatever custom RGB color she wants for the gnome's base glow.
- **Smart Animation:** Since the Arduino knows *exactly* when the clock is speaking (right inside your `announceTime()` function), it can trigger custom light animations on addressable LEDs (like Neopixels/WS2812B) automatically while the voice is playing, without needing any vibration sensors at all!

Whether you decide to keep it simple and just leave the lighting as a separate analog circuit or tie it into the new Arduino brain, that enclosure and gnome design are top-notch. It makes for such a meaningful, custom gift.

User prompt: I will leave the RGB like it is because it is only 5v powered, but if you can get the Smart Animation working that would be wonderfule.

Response: That is a smart plan—keeping the potentiometer color-mixing circuit totally independent keeps things simple and gives her immediate, tactile control over the glow whenever she wants it.

However, adding smart animations to the LEDs *while* the clock is talking is a fantastic way to bring the gnome to life. Since your Arduino Uno is already driving the HW-261 MP3 shield via SPI and managing the clock, we can easily add a single data pin for an addressable LED strip (like a WS2812B / NeoPixel ring or strand) right inside the gnome.

How the Smart Animation Works

Because your code already uses blocking playback (`musicPlayer.playFullFile()`), the Arduino pauses while the audio plays. We can use that exact window to trigger a cool animation (like a pulsing or sparkling effect) and then return the LEDs back to normal when the speech finishes!

Here is how you can integrate the Adafruit NeoPixel library into your current script to add automated animations during speech.

Updated Code with NeoPixel Animation

```
#include <SPI.h>
#include <SD.h>
#include <Wire.h>
#include "RTClib.h"
#include <Adafruit_VS1053.h>
#include <TM1637Display.h>
#include <Adafruit_NeoPixel.h>

// --- VERIFIED HW-261 PIN DEFINITIONS ---
#define VS1053_RST    8
#define VS1053_CS     6
#define VS1053_DCS    7
#define VS1053_DREQ   2
#define CARDCS        9

// --- BUTTON PINS ---
#define SAY_TIME_BTN  5
#define MODE_BTN      4
#define INC_BTN       10

// --- TM1637 DISPLAY PINS ---
#define CLK_PIN       A0
#define DIO_PIN       A1

// --- NEOPIXEL / LED STRIP CONFIGURATION ---
#define LED_PIN       A2    // Digital/Analog pin connected to the NeoPixel data line (DIN)
#define NUM_PIXELS    12    // Change this to match the number of LEDs in your gnome strip

Adafruit_VS1053_FilePlayer musicPlayer =
  Adafruit_VS1053_FilePlayer(VS1053_RST, VS1053_CS, VS1053_DCS, VS1053_DREQ, CARDCS);

RTC_DS3231 rtc;
TM1637Display display(CLK_PIN, DIO_PIN);
Adafruit_NeoPixel strip(NUM_PIXELS, LED_PIN, NEO_GRB + NEO_KHZ800);

// Clock States
enum ClockState { NORMAL, SET_HOUR, SET_MINUTE };
ClockState currentState = NORMAL;

int tempHour = 12;
int tempMinute = 0;
int lastMinuteChecked = -1;
unsigned long lastButtonPress = 0;
const unsigned long debounceDelay = 300;

void setup() {
  Serial.begin(9600);
  Serial.println(F("Initializing Talking Clock with LEDs..."));

  pinMode(SAY_TIME_BTN, INPUT_PULLUP);
  pinMode(MODE_BTN, INPUT_PULLUP);
  pinMode(INC_BTN, INPUT_PULLUP);
```

```
  display.setBrightness(0x0f);

  // Initialize NeoPixels
  strip.begin();
  strip.show(); // Initialize all pixels to 'off'
  setGnomeIdleColor(); // Set a soft welcome color

  if (!rtc.begin()) {
    Serial.println(F("Couldn't find RTC!"));
    while (1);
  }

  if (!musicPlayer.begin()) {
    Serial.println(F("Couldn't find VS1053!"));
    while (1);
  }

  if (!SD.begin(CARDCS)) {
    Serial.println(F("SD Card failed!"));
    while (1);
  }

  musicPlayer.setVolume(20, 20);
  musicPlayer.useInterrupt(VS1053_FILEPLAYER_PIN_INT);

  Serial.println(F("Ready!"));
}

void loop() {
  DateTime now = rtc.now();
  int currentMinute = now.minute();
  int currentHour = now.hour() % 12;
  if (currentHour == 0) currentHour = 12;

  // --- BUTTON & MODE LOGIC ---
  if (digitalRead(MODE_BTN) == LOW && (millis() - lastButtonPress > debounceDelay)) {
    lastButtonPress = millis();
    if (currentState == NORMAL) {
      currentState = SET_HOUR;
      tempHour = currentHour;
    } else if (currentState == SET_HOUR) {
      currentState = SET_MINUTE;
      tempMinute = now.minute();
    } else if (currentState == SET_MINUTE) {
      currentState = NORMAL;
      rtc.adjust(DateTime(now.year(), now.month(), now.day(), now.hour(), tempMinute, 0));
    }
  }

  if (digitalRead(INC_BTN) == LOW && (millis() - lastButtonPress > debounceDelay)) {
    lastButtonPress = millis();
    if (currentState == SET_HOUR) {
      tempHour++;
      if (tempHour > 12) tempHour = 1;
    } else if (currentState == SET_MINUTE) {
      tempMinute++;
      if (tempMinute > 59) tempMinute = 0;
    }
  }

  // --- NORMAL VS SETTING OPERATIONS ---
  if (currentState == NORMAL) {
    display.showNumberDecEx(currentHour * 100 + now.minute(), 0b01000000, true);
```

```
    // 15-Minute Automatic Check
    if ((currentMinute == 0 || currentMinute == 15 || currentMinute == 30 || currentMinute == 45)
        && currentMinute != lastMinuteChecked) {
      lastMinuteChecked = currentMinute;
      announceTimeWithAnimation(now.hour(), currentMinute);
    }

    if (currentMinute != 0 && currentMinute != 15 && currentMinute != 30 && currentMinute != 45) {
      lastMinuteChecked = -1;
    }

    // Manual Button Check
    if (digitalRead(SAY_TIME_BTN) == LOW) {
      if (millis() - lastButtonPress > debounceDelay) {
        lastButtonPress = millis();
        announceTimeWithAnimation(now.hour(), now.minute());
      }
    }
  }
  else if (currentState == SET_HOUR) {
    display.showNumberDecEx(tempHour * 100 + 0, 0b01000000, true);
  }
  else if (currentState == SET_MINUTE) {
    display.showNumberDecEx(tempHour * 100 + tempMinute, 0b01000000, true);
  }

  delay(100);
}

// --- AUDIO ANNOUNCEMENT WITH LED ANIMATION ---
void announceTimeWithAnimation(int hour, int minute) {
  // Trigger a fun sparkling or pulsing animation while speaking starts
  pulseGNOME();

  int displayHour = hour % 12;
  if (displayHour == 0) displayHour = 12;

  char audioFile[15];

  // Play Hour File
  sprintf(audioFile, "/h%d.mp3", displayHour);
  musicPlayer.playFullFile(audioFile);

  // Play Minute Files
  if (minute <= 20) {
    sprintf(audioFile, "/m%d.mp3", minute);
    musicPlayer.playFullFile(audioFile);
  }
  else {
    int tens = (minute / 10) * 10;
    int units = minute % 10;

    sprintf(audioFile, "/m%d.mp3", tens);
    musicPlayer.playFullFile(audioFile);

    if (units > 0) {
      sprintf(audioFile, "/u%d.mp3", units);
      musicPlayer.playFullFile(audioFile);
    }
  }

  // Return LEDs back to steady glow when speech finishes
  setGnomeIdleColor();
}
```

```
// --- LED ANIMATION FUNCTIONS ---
void setGnomeIdleColor() {
  // Soft steady purple/blue glow
  for (int i = 0; i < strip.numPixels(); i++) {
    strip.setPixelColor(i, strip.Color(75, 0, 130)); // Indigo / Purple
  }
  strip.show();
}

void pulseGNOME() {
  // Simple bright flash or pulse effect when it starts talking
  for (int b = 50; b <= 255; b += 50) {
    for (int i = 0; i < strip.numPixels(); i++) {
      strip.setPixelColor(i, strip.Color(0, b, b)); // Bright Cyan pulse
    }
    strip.show();
    delay(50);
  }
}
```

What This Adds:

1. **Pin `A2`** is now set as your NeoPixel data line. You can connect it directly to the `DIN` pin of a WS2812B data strip inside the gnome.
2. **`pulseGNOME()`** automatically fires the moment the clock begins speaking, creating a visual flash or pulse effect while the audio files are playing.
3. **`setGnomeIdleColor()`** automatically returns the gnome to a peaceful steady glow as soon as the voice finishes announcing the time.

User prompt: How do I change the LED colors and animation patterns when the talking clock speaks?

Response: Customizing the colors and animation patterns for your gnome is super easy! All the LED behavior is controlled right at the bottom of the sketch in two specific functions: `setGnomeIdleColor()` (for when the clock is quiet) and `pulseGNOME()` (for when the clock is speaking).

Here is how you can tweak them to get the exact look you want:

1. Understanding the Color Values (`strip.Color(Red, Green, Blue)`)

Every color is made by mixing **Red, Green, and Blue** on a scale from **0 to 255**:

- `strip.Color(255, 0, 0)` = Pure Red
- `strip.Color(0, 255, 0)` = Pure Green
- `strip.Color(0, 0, 255)` = Pure Blue
- `strip.Color(255, 255, 255)` = Bright White
- `strip.Color(255, 100, 0)` = Warm Orange / Gold

2. Changing the Idle Glow Color

To change what color the gnome glows normally when it is *not* talking, find this function in your code:

```
void setGnomeIdleColor() {
  // Currently set to a soft Indigo/Purple (75, 0, 130)
  for (int i = 0; i < strip.numPixels(); i++) {
    strip.setPixelColor(i, strip.Color(75, 0, 130));
  }
```

```
  strip.show();
}
```

- Simply change the numbers inside `strip.Color(75, 0, 130)` to whatever favorite color your wife wants for her steady background glow!

3. Changing the Spoken Animation Pattern

The `pulseGNOME()` function currently flashes a bright cyan pulse when the clock starts talking. You can replace or rewrite this function to do whatever kind of animation you want—like a sparkling effect, a rainbow flash, or a smooth fade.

Here are a few alternative animation styles you can swap in:

Option A: A Rapid Rainbow Sparkle

```
void pulseGNOME() {
  // Flashes random bright colors quickly while speaking starts
  for (int j = 0; j < 15; j++) { // Repeats 15 times
    for (int i = 0; i < strip.numPixels(); i++) {
      // Pick random red, green, blue values
      int r = random(50, 255);
      int g = random(50, 255);
      int b = random(50, 255);
      strip.setPixelColor(i, strip.Color(r, g, b));
    }
    strip.show();
    delay(100); // Speed of the sparkle
  }
}
```

Option B: A Warm Golden Fade In/Out

```
void pulseGNOME() {
  // Smoothly breathes a warm golden/orange glow twice
  for (int cycle = 0; cycle < 2; cycle++) {
    // Fade up
    for (int b = 20; b <= 255; b += 15) {
      for (int i = 0; i < strip.numPixels(); i++) {
        strip.setPixelColor(i, strip.Color(b, b / 2, 0)); // Warm gold
      }
      strip.show();
      delay(30);
    }
    // Fade down
    for (int b = 255; b >= 20; b -= 15) {
      for (int i = 0; i < strip.numPixels(); i++) {
        strip.setPixelColor(i, strip.Color(b, b / 2, 0));
      }
      strip.show();
      delay(30);
    }
  }
}
```

User prompt: Thanks Gemini, I got work to do

Response: You're very welcome! Have a great time working on the new base and getting everything wired up. Your wife is going to love the upgraded, bulletproof talking clock and the glowing gnome.

If you run into any questions while assembling the proto shield or loading up the code, just reach out. Good luck with the build!

User prompt: can you se why the hours or not saving? the minutes saved

Response: The hours aren't saving because of how the code handles the **24-hour format vs. 12-hour format** when saving back to the DS3231.

When you use the increment button, `tempHour` only goes from **1 to 12**, but the DS3231 expects a **0 to 23 (24-hour)** format. When you hit save, it passes that 1–12 value straight into the hour slot, which throws off the actual time (making it look like only the minutes saved).

The Fix

To fix this, we need to account for whether it is AM or PM when saving, or cleanly track the hour in 24-hour format during the setting state.

Here is the corrected snippet for your `MODE_BTN` save routine. Replace that specific block in your code with this:

```
    else if (currentState == SET_MINUTE) {
      currentState = NORMAL;

      // Figure out current AM/PM from the RTC before saving
      int current24Hour = now.hour();
      int finalHour = tempHour;

      // If it's currently PM (12:00 to 23:59) and they picked 1-11, add 12 to make it PM
      if (current24Hour >= 12 && tempHour < 12) {
        finalHour += 12;
      }
      // If it's currently AM and they picked 12, make it 00 (midnight)
      else if (current24Hour < 12 && tempHour == 12) {
        finalHour = 0;
      }
      // If it's PM and they picked 12, keep it 12 (noon)
      else if (current24Hour >= 12 && tempHour == 12) {
        finalHour = 12;
      }

      // Save new correct 24h time back to RTC
      rtc.adjust(DateTime(now.year(), now.month(), now.day(), finalHour, tempMinute, 0));
      Serial.println(F("Time Saved to RTC!"));
    }
```

Give that adjustment a try, and your hours will start saving correctly right alongside your minutes!

User prompt: Can you give me the full updated Arduino code with the hour-saving fix included?

Response: Here is the full, updated Arduino code with the 12-to-24 hour conversion fix fully integrated into the save routine so your hours and minutes will both save correctly!

```
#include <SPI.h>
#include <SD.h>
#include <Wire.h>
#include "RTClib.h"
#include <Adafruit_VS1053.h>
#include <TM1637Display.h>
#include <Adafruit_NeoPixel.h>

// --- VERIFIED HW-261 PIN DEFINITIONS ---
#define VS1053_RST    8
#define VS1053_CS     6
#define VS1053_DCS    7
#define VS1053_DREQ   2
#define CARDCS        9

// --- BUTTON PINS ---
```

```
#define SAY_TIME_BTN  5
#define MODE_BTN      4
#define INC_BTN       10

// --- TM1637 DISPLAY PINS ---
#define CLK_PIN       A0
#define DIO_PIN       A1

// --- NEOPIXEL / LED STRIP CONFIGURATION ---
#define LED_PIN       A2
#define NUM_PIXELS    12

Adafruit_VS1053_FilePlayer musicPlayer =
  Adafruit_VS1053_FilePlayer(VS1053_RST, VS1053_CS, VS1053_DCS, VS1053_DREQ, CARDCS);

RTC_DS3231 rtc;
TM1637Display display(CLK_PIN, DIO_PIN);
Adafruit_NeoPixel strip(NUM_PIXELS, LED_PIN, NEO_GRB + NEO_KHZ800);

// Clock States
enum ClockState { NORMAL, SET_HOUR, SET_MINUTE };
ClockState currentState = NORMAL;

int tempHour = 12;
int tempMinute = 0;
int lastMinuteChecked = -1;
unsigned long lastButtonPress = 0;
const unsigned long debounceDelay = 300;

void setup() {
  Serial.begin(9600);
  Serial.println(F("Initializing Talking Clock with Fixed Hour Save..."));

  pinMode(SAY_TIME_BTN, INPUT_PULLUP);
  pinMode(MODE_BTN, INPUT_PULLUP);
  pinMode(INC_BTN, INPUT_PULLUP);

  display.setBrightness(0x0f);

  // Initialize NeoPixels
  strip.begin();
  strip.show();
  setGnomeIdleColor();

  if (!rtc.begin()) {
    Serial.println(F("Couldn't find RTC!"));
    while (1);
  }

  if (!musicPlayer.begin()) {
    Serial.println(F("Couldn't find VS1053!"));
    while (1);
  }

  if (!SD.begin(CARDCS)) {
    Serial.println(F("SD Card failed!"));
    while (1);
  }

  musicPlayer.setVolume(20, 20);
  musicPlayer.useInterrupt(VS1053_FILEPLAYER_PIN_INT);

  Serial.println(F("Ready!"));
}
```

```
void loop() {
  DateTime now = rtc.now();
  int currentMinute = now.minute();
  int currentHour = now.hour() % 12;
  if (currentHour == 0) currentHour = 12;

  // --- BUTTON & MODE LOGIC ---
  if (digitalRead(MODE_BTN) == LOW && (millis() - lastButtonPress > debounceDelay)) {
    lastButtonPress = millis();
    if (currentState == NORMAL) {
      currentState = SET_HOUR;
      tempHour = currentHour;
      Serial.println(F("Mode: Setting Hour"));
    }
    else if (currentState == SET_HOUR) {
      currentState = SET_MINUTE;
      tempMinute = now.minute();
      Serial.println(F("Mode: Setting Minute"));
    }
    else if (currentState == SET_MINUTE) {
      currentState = NORMAL;

      // FIX: Convert 12h user input back into proper 24h format for DS3231
      int current24Hour = now.hour();
      int finalHour = tempHour;

      if (current24Hour >= 12 && tempHour < 12) {
        finalHour += 12; // PM adjustment
      }
      else if (current24Hour < 12 && tempHour == 12) {
        finalHour = 0;   // Midnight adjustment
      }
      else if (current24Hour >= 12 && tempHour == 12) {
        finalHour = 12;  // Noon adjustment
      }

      rtc.adjust(DateTime(now.year(), now.month(), now.day(), finalHour, tempMinute, 0));
      Serial.println(F("Time Saved to RTC!"));
    }
  }

  if (digitalRead(INC_BTN) == LOW && (millis() - lastButtonPress > debounceDelay)) {
    lastButtonPress = millis();
    if (currentState == SET_HOUR) {
      tempHour++;
      if (tempHour > 12) tempHour = 1;
    } else if (currentState == SET_MINUTE) {
      tempMinute++;
      if (tempMinute > 59) tempMinute = 0;
    }
  }

  // --- NORMAL VS SETTING OPERATIONS ---
  if (currentState == NORMAL) {
    display.showNumberDecEx(currentHour * 100 + now.minute(), 0b01000000, true);

    // 15-Minute Automatic Check
    if ((currentMinute == 0 || currentMinute == 15 || currentMinute == 30 || currentMinute == 45)
        && currentMinute != lastMinuteChecked) {
      lastMinuteChecked = currentMinute;
      announceTimeWithAnimation(now.hour(), currentMinute);
    }
```

```
    if (currentMinute != 0 && currentMinute != 15 && currentMinute != 30 && currentMinute != 45) {
      lastMinuteChecked = -1;
    }

    // Manual Button Check
    if (digitalRead(SAY_TIME_BTN) == LOW) {
      if (millis() - lastButtonPress > debounceDelay) {
        lastButtonPress = millis();
        announceTimeWithAnimation(now.hour(), now.minute());
      }
    }
  }
  else if (currentState == SET_HOUR) {
    display.showNumberDecEx(tempHour * 100 + 0, 0b01000000, true);
  }
  else if (currentState == SET_MINUTE) {
    display.showNumberDecEx(tempHour * 100 + tempMinute, 0b01000000, true);
  }

  delay(100);
}

// --- AUDIO ANNOUNCEMENT WITH LED ANIMATION ---
void announceTimeWithAnimation(int hour, int minute) {
  pulseGNOME();

  int displayHour = hour % 12;
  if (displayHour == 0) displayHour = 12;

  char audioFile[15];

  // Play Hour File
  sprintf(audioFile, "/h%d.mp3", displayHour);
  musicPlayer.playFullFile(audioFile);

  // Play Minute Files
  if (minute <= 20) {
    sprintf(audioFile, "/m%d.mp3", minute);
    musicPlayer.playFullFile(audioFile);
  }
  else {
    int tens = (minute / 10) * 10;
    int units = minute % 10;

    sprintf(audioFile, "/m%d.mp3", tens);
    musicPlayer.playFullFile(audioFile);

    if (units > 0) {
      sprintf(audioFile, "/u%d.mp3", units);
      musicPlayer.playFullFile(audioFile);
    }
  }

  setGnomeIdleColor();
}

// --- LED ANIMATION FUNCTIONS ---
void setGnomeIdleColor() {
  for (int i = 0; i < strip.numPixels(); i++) {
    strip.setPixelColor(i, strip.Color(75, 0, 130)); // Indigo / Purple
  }
  strip.show();
}
```

```
void pulseGNOME() {
  for (int b = 50; b <= 255; b += 50) {
    for (int i = 0; i < strip.numPixels(); i++) {
      strip.setPixelColor(i, strip.Color(0, b, b)); // Bright Cyan pulse
    }
    strip.show();
    delay(50);
  }
}
```

User prompt: that fixed it! thanks gemini

Response: That is awesome news! I'm so glad the 12-to-24 hour conversion fix got the setting menu working smoothly.

You've got a seriously cool project coming together here—combining the reliable standalone HW-261 audio, the DS3231 RTC, custom voice files, the TM1637 display, and those animated gnome LEDs makes for an incredible build.

Have a wonderful time putting together the new base and soldering up those JST harnesses. If you need any more tweaks or run into anything else while finishing it up, just reach out. Good luck with the build!

User prompt: Thanks Gemini, her clock is back on show.

Response: That looks incredible! Seeing it fully assembled, sitting proudly on display with the TM1637 display glowing, the buttons lined up neatly, and that gorgeous purple light radiating through the gnome is an absolute triumph.

You brought that whole build together brilliantly—from solving the audio and code hurdles to packing it all into that custom enclosure. It looks right at home in its spot.

Enjoy the finished clock, and let me know if you ever decide to tackle another custom electronics project!

User prompt: would it be hard to have the clock play a mp3 sound, like at 6 :01 Am , the other clock played a rooster crowing at 601

Response: Adding a special sound effect like a rooster crowing at a specific time (like 6:01 AM) is actually very easy to add into your code!

Since your Arduino already knows the exact hour and minute from the DS3231, you just need to drop a special audio file onto your SD card (for example, `rooster.mp3`) and add a quick check inside your `loop()` function.

How to Add It to Your Code

You can add a specific time-check right alongside your 15-minute interval check. Here is how you can structure it:

```
// --- Check for 6:01 AM Rooster Alarm ---
if (now.hour() == 6 && currentMinute == 1 && currentMinute != lastMinuteChecked) {
  lastMinuteChecked = currentMinute;

  // Optional: Play the rooster sound instead of or before the time
  musicPlayer.playFullFile("/rooster.mp3");

  // If you want it to also speak the time right after the rooster crows:
  announceTimeWithAnimation(now.hour(), currentMinute);
}
```

What You Need on the SD Card:

1. Make sure **`rooster.mp3`** is placed directly in the root directory of your MicroSD card.
2. The code above will catch the exact moment it turns **6:01 AM**, play the rooster sound file, and optionally announce the time right after.

User prompt: Awesome that worked, I set it to test and it went off. the only issue is, it keeps playing over and over untin the minute is up

Response: Ah, that is a classic catch! Because the Arduino's `loop()` runs many times a second, as long as the clock reads **6:01**, it sees that `currentMinute == 1` and triggers the alarm over and over again for the full 60 seconds until the clock ticks over to 6:02.

To fix that, we just need a dedicated tracking variable—similar to how we handle `lastMinuteChecked`—so the alarm file **only triggers once** when the minute first hits 1, and won't trigger again until tomorrow morning.

The Fix

Here is how you can update your automatic check block in the `loop()` to prevent it from repeating:

1. Add a tracking variable at the top of your sketch (near `lastMinuteChecked`):

```
int lastRoosterDay = -1; // Tracks the last day the rooster crowed
```

2. Update your time check inside `loop()` to verify it's 6:01 **and** that it hasn't already played today:

```
  // --- 6:01 AM ROOSTER ALARM CHECK ---
  if (now.hour() == 6 && currentMinute == 1) {
    // Check if we haven't already played it today (using the day of the month)
    if (now.day() != lastRoosterDay) {
      lastRoosterDay = now.day(); // Lock it out for the rest of today

      Serial.println(F("Good morning! Playing rooster alarm..."));
      musicPlayer.playFullFile("/rooster.mp3");

      // Optional: Follow up with the spoken time
      announceTimeWithAnimation(now.hour(), currentMinute);
    }
  }
```

By checking `now.day() != lastRoosterDay`, it plays precisely once when 6:01 hits, locks out for the rest of the day, and automatically resets itself ready for the next morning when the date changes!

User prompt: that worked, only plays once. I was thinking, how does it know 6 am from 6 pm? but if it dows play at 6 pm also that ok.

Response: That is a great question!

Because the DS3231 real-time clock stores time in a 24-hour format under the hood, **`now.hour()` actually distinguishes between AM and PM**:

- **6:01 AM** is hour `6`
- **6:01 PM** is hour `18`

Since the code specifically checks for `now.hour() == 6`, it will **only** trigger at 6:01 AM and will completely ignore 6:01 PM!

However, if you *do* want it to crow at both 6:01 AM and 6:01 PM, you can easily update the check using the logical OR operator (`||`) like this:

```
if ((now.hour() == 6 || now.hour() == 18) && currentMinute == 1) {
  // ... plays the rooster alarm
}
```

(If you use that, you'd just want to track the hour or use a slightly different lockout so it can play once in the morning and once in the evening).

Otherwise, keeping it at `now.hour() == 6` ensures it stays strictly as a morning wake-up call!

User prompt: well reason for question is, it is currently 3:45 PM and I tested the roster for hour 3 and minute 45 and it crowed, maybe because it is set for 12 hour format?

Response: You hit the nail right on the head! That is exactly why it happened.

Even though the DS3231 stores time in 24-hour format under the hood, your test used `now.hour() == 6`, but since you configured your display and setting menus around a **12-hour clock**, if your clock was currently set to **3:45 PM**, `now.hour()` was actually returning `15` (24-hour time).

If it crowed when you tested it at 3:45 PM, it means your DS3231 was currently sitting at **3:01 AM** (where `now.hour()` equals `3`)!

Since the DS3231 doesn't know about AM/PM on its own (it just counts 0 to 23 hours), if you want the alarm to trigger at a specific time, you have to account for whether it's AM or PM in the code, or set the clock using 24-hour time.

If you want to make sure the rooster *only* crows at **6:00 AM** and never at 6:00 PM, you can check both the hour *and* ensure it's morning, or you can check if `now.hour() == 6`. But if it crowed at 3:45 PM during your test, your RTC's internal 24-hour clock was likely in the AM cycle (around 3:00 AM) when you pressed the test condition.

User prompt: how to add a am/pm check, with a led ?

Response: To add an explicit **AM/PM check** and tie it to an indicator LED (so you can tell at a glance whether the clock thinks it is morning or afternoon), it is actually very straightforward!

Since the DS3231 stores time in 24-hour format (`0` to `23`), **AM** is simply when the hour is between `0` and `11`, and **PM** is when the hour is between `12` and `23`.

1. Wiring an AM/PM LED

1. Take an LED and connect its **long leg (anode)** through a 220-ohm resistor to any open digital pin (for example, **Digital Pin 3**).
2. Connect the **short leg (cathode)** to any **GND** pin.

2. Updated Code with AM/PM Tracking & LED

Here is how you add a pin definition for the LED, update your loop to light it up whenever it is PM (or AM, depending on preference), and ensure your 6:01 AM rooster alarm *strictly* happens in the morning:

```
// --- ADD THIS PIN DEFINITION AT THE TOP ---
#define AMPM_LED_PIN  3    // Digital pin for your AM/PM indicator LED (HIGH = PM, LOW = AM)
```

Inside your `setup()` function, make sure to configure it as an output:

```
  pinMode(AMPM_LED_PIN, OUTPUT);
```

Inside your `loop()`, you can update your time checks and LED status like this:

```
  // --- AM/PM LED STATUS CHECK ---
  // If hour is 12 (Noon) or greater, it's PM (turn LED ON). Otherwise, it's AM (turn LED OFF).
  if (now.hour() >= 12) {
    digitalWrite(AMPM_LED_PIN, HIGH); // LED ON = PM
  } else {
    digitalWrite(AMPM_LED_PIN, LOW);  // LED OFF = AM
  }

  // --- 6:01 AM STRICT MORNING ROOSTER ALARM CHECK ---
  // now.hour() == 6 ensures it is specifically 6:01 AM (and not 6:01 PM which would be hour 18)
  if (now.hour() == 6 && currentMinute == 1) {
    if (now.day() != lastRoosterDay) {
      lastRoosterDay = now.day();

      Serial.println(F("Good morning! Playing rooster alarm..."));
      musicPlayer.playFullFile("/rooster.mp3");
```

```
      announceTimeWithAnimation(now.hour(), currentMinute);
    }
  }
```

How this solves it:

- **For the Alarm:** Because `now.hour()` strictly equals `6` only during the morning hours (6:00 AM to 6:59 AM), evening hours (which are `18` for 6 PM) will never trigger the rooster.
- **For the LED:** You now have a visual indicator. If you ever look at the clock in the afternoon and notice the LED is off, you'll instantly know if the internal 24-hour clock got out of sync!

User prompt: Thanks! I have a thought, to save me having to add the led, would it be easer to have the time setting mode in 24 hour format? or is it simpler to add a led.

Response: Switching the time-setting mode to a **24-hour format** is actually much easier than adding an extra LED, because it completely eliminates any guesswork about AM vs. PM behind the scenes!

By letting you set the clock directly from 00:00 to 23:59, the Arduino's internal 24-hour clock (`now.hour()`) will always perfectly match reality. This means your 6:01 AM rooster alarm will fire strictly at 6 AM, and it will never accidentally go off at 6 PM.

How to Switch to 24-Hour Time Setting

To change your setting menu to 24-hour format, you just need to adjust your temporary hour limits and remove the AM/PM conversion logic during the save step.

Here are the specific changes to make in your code:

1. **Change the Increment Limit for Hours:** When setting the hour, instead of looping from 1 to 12, let it loop from **0 to 23**:

   ```
   if (currentState == SET_HOUR) {
     tempHour++;
     if (tempHour > 23) tempHour = 0; // Loops from 0 to 23 hours
   }
   ```

2. **Simplify the Save Step:** Since `tempHour` is already in 24-hour format, you don't need any complex AM/PM math. You just save it directly:

   ```
   else if (currentState == SET_MINUTE) {
     currentState = NORMAL;

     // Directly save the 24-hour time to the RTC
     rtc.adjust(DateTime(now.year(), now.month(), now.day(), tempHour, tempMinute, 0));
     Serial.println(F("24-Hour Time Saved to RTC!"));
   }
   ```

What about the Display?

Your TM1637 display will still show whatever numbers you pass to it. If you want, you can display it in 24-hour format (e.g., `18:30` for 6:30 PM), or you can keep the display showing a 12-hour format while the background setting mode uses 24-hour time.

Skipping the physical LED wiring and just letting the system natively use 24-hour time is definitely the cleaner, simpler software solution!

User prompt: Can you give me the full Arduino code updated with the 24-hour time setting mode?

Response: Here is your full, complete Arduino code updated to use a **24-hour time setting mode** (0 to 23 hours). This completely eliminates any AM/PM confusion, keeps your background RTC tracking accurate, and ensures your 6:01 AM rooster alarm will fire precisely when intended without needing an extra indicator LED!

```
#include <SPI.h>
#include <SD.h>
```

```
#include <Wire.h>
#include "RTClib.h"
#include <Adafruit_VS1053.h>
#include <TM1637Display.h>
#include <Adafruit_NeoPixel.h>

// --- VERIFIED HW-261 PIN DEFINITIONS ---
#define VS1053_RST    8
#define VS1053_CS     6
#define VS1053_DCS    7
#define VS1053_DREQ   2
#define CARDCS        9

// --- BUTTON PINS ---
#define SAY_TIME_BTN  5
#define MODE_BTN      4
#define INC_BTN       10

// --- TM1637 DISPLAY PINS ---
#define CLK_PIN       A0
#define DIO_PIN       A1

// --- NEOPIXEL / LED STRIP CONFIGURATION ---
#define LED_PIN       A2
#define NUM_PIXELS    12

Adafruit_VS1053_FilePlayer musicPlayer =
  Adafruit_VS1053_FilePlayer(VS1053_RST, VS1053_CS, VS1053_DCS, VS1053_DREQ, CARDCS);

RTC_DS3231 rtc;
TM1637Display display(CLK_PIN, DIO_PIN);
Adafruit_NeoPixel strip(NUM_PIXELS, LED_PIN, NEO_GRB + NEO_KHZ800);

// Clock States
enum ClockState { NORMAL, SET_HOUR, SET_MINUTE };
ClockState currentState = NORMAL;

int tempHour = 12;
int tempMinute = 0;
int lastMinuteChecked = -1;
int lastRoosterDay = -1; // Tracks the last day the rooster crowed
unsigned long lastButtonPress = 0;
const unsigned long debounceDelay = 300;

void setup() {
  Serial.begin(9600);
  Serial.println(F("Initializing Talking Clock (24-Hour Setting Mode)..."));

  pinMode(SAY_TIME_BTN, INPUT_PULLUP);
  pinMode(MODE_BTN, INPUT_PULLUP);
  pinMode(INC_BTN, INPUT_PULLUP);

  display.setBrightness(0x0f);

  // Initialize NeoPixels
  strip.begin();
  strip.show();
  setGnomeIdleColor();

  if (!rtc.begin()) {
    Serial.println(F("Couldn't find RTC!"));
    while (1);
  }
```

```
  if (!musicPlayer.begin()) {
    Serial.println(F("Couldn't find VS1053!"));
    while (1);
  }

  if (!SD.begin(CARDCS)) {
    Serial.println(F("SD Card failed!"));
    while (1);
  }

  musicPlayer.setVolume(20, 20);
  musicPlayer.useInterrupt(VS1053_FILEPLAYER_PIN_INT);

  Serial.println(F("Ready!"));
}

void loop() {
  DateTime now = rtc.now();
  int currentMinute = now.minute();
  int currentHour24 = now.hour(); // Get true 24h hour from RTC

  // --- BUTTON & MODE LOGIC ---
  if (digitalRead(MODE_BTN) == LOW && (millis() - lastButtonPress > debounceDelay)) {
    lastButtonPress = millis();
    if (currentState == NORMAL) {
      currentState = SET_HOUR;
      tempHour = currentHour24; // Start editing from current 24h hour
      Serial.println(F("Mode: Setting Hour (24h)"));
    }
    else if (currentState == SET_HOUR) {
      currentState = SET_MINUTE;
      tempMinute = now.minute();
      Serial.println(F("Mode: Setting Minute"));
    }
    else if (currentState == SET_MINUTE) {
      currentState = NORMAL;

      // Directly save the 24-hour time to the RTC
      rtc.adjust(DateTime(now.year(), now.month(), now.day(), tempHour, tempMinute, 0));
      Serial.println(F("24-Hour Time Saved to RTC!"));
    }
  }

  if (digitalRead(INC_BTN) == LOW && (millis() - lastButtonPress > debounceDelay)) {
    lastButtonPress = millis();
    if (currentState == SET_HOUR) {
      tempHour++;
      if (tempHour > 23) tempHour = 0; // Loop 0 to 23
    } else if (currentState == SET_MINUTE) {
      tempMinute++;
      if (tempMinute > 59) tempMinute = 0;
    }
  }

  // --- NORMAL VS SETTING OPERATIONS ---
  if (currentState == NORMAL) {
    // Show 12-hour format on display for normal viewing (converting 24h to 12h for the display numbers)
    int displayHour = currentHour24 % 12;
    if (displayHour == 0) displayHour = 12;
    display.showNumberDecEx(displayHour * 100 + now.minute(), 0b01000000, true);

    // --- 6:01 AM STRICT MORNING ROOSTER ALARM CHECK ---
    if (currentHour24 == 6 && currentMinute == 1) {
      if (now.day() != lastRoosterDay) {
```

```
        lastRoosterDay = now.day();

        Serial.println(F("Good morning! Playing rooster alarm..."));
        musicPlayer.playFullFile("/rooster.mp3");
        announceTimeWithAnimation(currentHour24, currentMinute);
      }
    }

    // 15-Minute Automatic Check
    if ((currentMinute == 0 || currentMinute == 15 || currentMinute == 30 || currentMinute == 45)
        && currentMinute != lastMinuteChecked) {
      lastMinuteChecked = currentMinute;
      announceTimeWithAnimation(currentHour24, currentMinute);
    }

    if (currentMinute != 0 && currentMinute != 15 && currentMinute != 30 && currentMinute != 45) {
      lastMinuteChecked = -1;
    }

    // Manual Button Check
    if (digitalRead(SAY_TIME_BTN) == LOW) {
      if (millis() - lastButtonPress > debounceDelay) {
        lastButtonPress = millis();
        announceTimeWithAnimation(currentHour24, currentMinute);
      }
    }
  }
  else if (currentState == SET_HOUR) {
    // Display the 24-hour hour value while setting (e.g., 1400 for 2 PM)
    display.showNumberDecEx(tempHour * 100 + 0, 0b01000000, true);
  }
  else if (currentState == SET_MINUTE) {
    display.showNumberDecEx(tempHour * 100 + tempMinute, 0b01000000, true);
  }

  delay(100);
}

// --- AUDIO ANNOUNCEMENT WITH LED ANIMATION ---
void announceTimeWithAnimation(int hour24, int minute) {
  pulseGNOME();

  int displayHour = hour24 % 12;
  if (displayHour == 0) displayHour = 12;

  char audioFile[15];

  // Play Hour File
  sprintf(audioFile, "/h%d.mp3", displayHour);
  musicPlayer.playFullFile(audioFile);

  // Play Minute Files
  if (minute <= 20) {
    sprintf(audioFile, "/m%d.mp3", minute);
    musicPlayer.playFullFile(audioFile);
  }
  else {
    int tens = (minute / 10) * 10;
    int units = minute % 10;

    sprintf(audioFile, "/m%d.mp3", tens);
    musicPlayer.playFullFile(audioFile);

    if (units > 0) {
```

```
      sprintf(audioFile, "/u%d.mp3", units);
      musicPlayer.playFullFile(audioFile);
    }
  }

  setGnomeIdleColor();
}

// --- LED ANIMATION FUNCTIONS ---
void setGnomeIdleColor() {
  for (int i = 0; i < strip.numPixels(); i++) {
    strip.setPixelColor(i, strip.Color(75, 0, 130)); // Indigo / Purple
  }
  strip.show();
}

void pulseGNOME() {
  for (int b = 50; b <= 255; b += 50) {
    for (int i = 0; i < strip.numPixels(); i++) {
      strip.setPixelColor(i, strip.Color(0, b, b)); // Bright Cyan pulse
    }
    strip.show();
    delay(50);
  }
}
```

User prompt: ok when I go to set time, it still 12 hours

Response: Ah, I see why! Even though the code is counting up to 23, when it displays the hour during the `SET_HOUR` state, it was still using `displayHour` or wrapping around.

In that section, look at this part of the code:

```
  else if (currentState == SET_HOUR) {
    // Display the 24-hour hour value while setting
    display.showNumberDecEx(tempHour * 100 + 0, 0b01000000, true);
  }
```

Wait, if `tempHour` goes from 0 to 23, it *should* display numbers like `1300`, `1400`, up to `2300`. But if you want it to be even more intuitive so you can see the exact 24-hour number you are setting (like `14` for 2 PM), let's make sure `tempHour` is incrementing properly up to 23.

Check your increment button logic for `SET_HOUR`:

```
    if (currentState == SET_HOUR) {
      tempHour++;
      if (tempHour > 23) tempHour = 0; // Make sure this is 23, not 12!
    }
```

If it still looks like it's sticking to 12 hours, double-check that you uploaded the newest script to your Arduino, as it's easy for the IDE to hold onto an older version in memory.

Let me know if `tempHour` is still looping at 12 or if it's letting you click past it!

User prompt: the new code is still 12 hours , no 13:00 - 23:59

Response: Ah, I see what is happening! When you enter `SET_HOUR`, the code initializes `tempHour` using this line:

```
tempHour = currentHour24;
```

If you happened to be setting the time while `currentHour24` was under 12, or if there's any other place forcing it, let's look at how `tempHour` increments and resets.

To make absolutely sure it rolls all the way from `0` to `23` without getting stuck, check the increment button block in your loop. Make sure it looks exactly like this:

```
  if (digitalRead(INC_BTN) == LOW && (millis() - lastButtonPress > debounceDelay)) {
    lastButtonPress = millis();
    if (currentState == SET_HOUR) {
      tempHour++;
      if (tempHour > 23) {
        tempHour = 0; // Wraps from 23 back to midnight (0)
      }
    } else if (currentState == SET_MINUTE) {
      tempMinute++;
      if (tempMinute > 59) tempMinute = 0;
    }
  }
```

Also, when you enter `SET_HOUR`, if you want to start editing right from whatever the clock currently reads (in 24-hour time), ensure your `MODE_BTN` block has this:

```
    if (currentState == NORMAL) {
      currentState = SET_HOUR;
      tempHour = rtc.now().hour(); // Grabs exact 24h hour from RTC right now
      Serial.println(F("Mode: Setting Hour (24h)"));
    }
```

Double-check those two blocks in your sketch. If `tempHour` goes past 12, it will proudly display `1300`, `1400`, all the way up to `2300` on your TM1637 display!

User prompt: that worked! it's 4:30 pm and it let me set time to 16:30

Response: That is awesome! Seeing **16:30** pop up on the display means the 24-hour mode is locked in and working like a charm.

Now your clock knows the exact difference between morning and evening, your 6:01 AM rooster alarm will fire right on schedule without any afternoon surprises, and your setting menu is completely foolproof.

How is the physical assembly coming along with your proto shield and JST connectors?

User prompt: JST connector, proto board worked fine! I did have to add a filter for the audio, I learned that from you from another project.

Response: That internal setup looks like an absolute work of art! The stacked proto shield with the neat white JST connectors, the clean power management board with the LM2596 step-down regulator, and that custom-filtered audio breakout board with the magnetic core/capacitor filtering show some serious maker craftsmanship.

Using an inline audio filter to clean up any power supply hum or digital noise going into the amplifier makes a world of difference for voice clarity—especially when driving a small speaker inside a hollow enclosure.

It looks wonderfully secure, professional, and ready for many years of trouble-free 15-minute announcements and morning rooster alarms. Your wife is going to love having her custom gnome clock back up and running in top shape!

If you ever decide to build another gadget or tweak anything else down the road, you know where to find me. Enjoy the finished build!

User prompt: I just tested for 4:50 PM 22:50 and the rooster did not crow! so alarm is Smart now. only crows in morning

Response: That means your alarm logic is officially **spot on**!

By shifting the system to a clean 24-hour setting mode, `currentHour24 == 6` strictly locks the rooster alarm into the morning hours, leaving your afternoon tests (like 16:50) peacefully quiet.

Your talking clock is fully intelligent, reliable, and ready for daily service. Enjoy watching your wife's reaction to having her custom gnome clock back up and running better than ever!

User prompt: I will post our work on github for other to follow and build, except I will show a simple clock case instead of the gnome thing. maybe just a 16 LED ring.

Response: That is a fantastic idea! Sharing the project on GitHub will be a huge help to other makers looking for a reliable, non-Wi-Fi talking clock build. Swapping out the gnome for a sleek simple case with a 16-LED NeoPixel ring gives it a very modern, desktop-friendly look that anyone can easily print and assemble.

If you end up swapping in a 16-LED ring, you can easily update your `NUM_PIXELS` define at the top of your code:

```
#define NUM_PIXELS    16    // Updated for a 16-LED NeoPixel ring
```

And if you want to make the ring do a cool smooth circle animation when the clock speaks instead of a simple flash, a quick loading animation looks amazing on a ring!

Thank you for letting me tag along on this project from the initial pin mapping all the way to the final polished build. Your documentation is going to save a lot of people from flaky Wi-Fi clocks!

User prompt: a GitHub README would be awsome!

Response: Here is a clean, comprehensive GitHub README template tailored for your project. You can copy and paste this directly into a `README.md` file in your repository!

Arduino Talking Clock with DS3231, MP3 Shield (HW-261), and LED Ring

A reliable, fully standalone desktop talking clock powered by an Arduino Uno. Unlike network-dependent clocks that drop connections, this build uses a DS3231 Real-Time Clock module for accurate timekeeping, a VS1053 MP3 shield for crystal-clear voice announcements, a TM1637 display, and addressable NeoPixel LEDs.

It automatically announces the time every 15 minutes, features a manual "Say Time" button, a 24-hour time-setting menu, and a dedicated morning rooster alarm at 6:01 AM!

Features

- **Standalone Timekeeping:** Powered by a DS3231 RTC (with coin cell battery backup). No Wi-Fi or internet connection required.
- **Dynamic Voice Synthesis:** Combines hour, tens, and unit MP3 files dynamically on the fly to speak any time of day.
- **Hourly & Manual Announcements:** Automatically chimes/speaks on the 15-minute marks (00, 15, 30, 45) and speaks on demand when a button is pressed.
- **Morning Rooster Alarm:** Plays a custom `rooster.mp3` alarm strictly at 6:01 AM (ignoring PM hours thanks to 24-hour time logic).
- **Time-Setting Menu:** Easily adjust hours and minutes directly on the hardware using dedicated mode and increment buttons.
- **Visual Feedback:** Supports a TM1637 4-digit display and NeoPixel LED lighting animations that pulse while the clock speaks.

Hardware Components

- **Arduino Uno** (or compatible board)
- **HW-261 VS1053 MP3 Shield** (with MicroSD card)
- **DS3231 RTC Module**

- **TM1637 4-digit 7-segment Display**
- **Addressable LED Ring / Strip** (e.g., WS2812B / NeoPixel)
- **PAM8403 Audio Amplifier Module** + 3W Speaker (plus inline audio filtering for clean sound)
- **3x Push Buttons** (Say Time, Mode, Increment)
- **JST Connectors & Proto Shield** (for clean internal wiring harness)

Pin Mapping

Component	Pin on Arduino Uno
VS1053 Reset	Pin 8
VS1053 CS	Pin 6
VS1053 DCS	Pin 7
VS1053 DREQ	Pin 2
SD Card CS	Pin 9
Say Time Button	Pin 5 (to GND with internal pull-up)
Mode Button	Pin 4 (to GND with internal pull-up)
Increment Button	Pin 10 (to GND with internal pull-up)
TM1637 CLK	Analog Pin A0
TM1637 DIO	Analog Pin A1
NeoPixel DIN	Analog Pin A2
DS3231 RTC	SDA (`A4`) & SCL (`A5`)

MicroSD Card Structure

Format your MicroSD card to **FAT32**. Place all audio files directly in the **root directory** using lowercase 8.3 naming conventions:

- **Hours:** `h1.mp3` through `h12.mp3`
- **Base Minutes & Tens:** `m0.mp3` (O'Clock), `m1.mp3` through `m20.mp3`, `m30.mp3`, `m40.mp3`, `m50.mp3`
- **Units (for 21–59):** `u1.mp3` through `u9.mp3`
- **Alarm:** `rooster.mp3`

Required Arduino Libraries

Make sure you have the following libraries installed via the Arduino IDE Library Manager:

- `Adafruit_VS1053`
- `RTClib` (by Adafruit)
- `TM1637Display` (by Avishay Orpaz)
- `Adafruit_NeoPixel`
- `SD` (Built-in)
- `SPI` (Built-in)

- `Wire` (Built-in)

Arduino Code

```
#include <SPI.h>
#include <SD.h>
#include <Wire.h>
#include "RTClib.h"
#include <Adafruit_VS1053.h>
#include <TM1637Display.h>
#include <Adafruit_NeoPixel.h>

// --- HW-261 PIN DEFINITIONS ---
#define VS1053_RST    8
#define VS1053_CS     6
#define VS1053_DCS    7
#define VS1053_DREQ   2
#define CARDCS        9

// --- BUTTON PINS ---
#define SAY_TIME_BTN  5
#define MODE_BTN      4
#define INC_BTN       10

// --- TM1637 DISPLAY PINS ---
#define CLK_PIN       A0
#define DIO_PIN       A1

// --- NEOPIXEL CONFIGURATION ---
#define LED_PIN       A2
#define NUM_PIXELS    16    // Adjust depending on your ring/strip size

Adafruit_VS1053_FilePlayer musicPlayer =
  Adafruit_VS1053_FilePlayer(VS1053_RST, VS1053_CS, VS1053_DCS, VS1053_DREQ, CARDCS);

RTC_DS3231 rtc;
TM1637Display display(CLK_PIN, DIO_PIN);
Adafruit_NeoPixel strip(NUM_PIXELS, LED_PIN, NEO_GRB + NEO_KHZ800);

// Clock States
enum ClockState { NORMAL, SET_HOUR, SET_MINUTE };
ClockState currentState = NORMAL;

int tempHour = 12;
int tempMinute = 0;
int lastMinuteChecked = -1;
int lastRoosterDay = -1;
unsigned long lastButtonPress = 0;
const unsigned long debounceDelay = 300;

void setup() {
  Serial.begin(9600);

  pinMode(SAY_TIME_BTN, INPUT_PULLUP);
  pinMode(MODE_BTN, INPUT_PULLUP);
  pinMode(INC_BTN, INPUT_PULLUP);

  display.setBrightness(0x0f);

  strip.begin();
  strip.show();
  setGnomeIdleColor();

  if (!rtc.begin()) while (1);
```

```
  if (!musicPlayer.begin()) while (1);
  if (!SD.begin(CARDCS)) while (1);

  musicPlayer.setVolume(20, 20);
  musicPlayer.useInterrupt(VS1053_FILEPLAYER_PIN_INT);
}

void loop() {
  DateTime now = rtc.now();
  int currentMinute = now.minute();
  int currentHour24 = now.hour();

  // --- BUTTON & MODE LOGIC ---
  if (digitalRead(MODE_BTN) == LOW && (millis() - lastButtonPress > debounceDelay)) {
    lastButtonPress = millis();
    if (currentState == NORMAL) {
      currentState = SET_HOUR;
      tempHour = currentHour24;
    }
    else if (currentState == SET_HOUR) {
      currentState = SET_MINUTE;
      tempMinute = now.minute();
    }
    else if (currentState == SET_MINUTE) {
      currentState = NORMAL;
      rtc.adjust(DateTime(now.year(), now.month(), now.day(), tempHour, tempMinute, 0));
    }
  }

  if (digitalRead(INC_BTN) == LOW && (millis() - lastButtonPress > debounceDelay)) {
    lastButtonPress = millis();
    if (currentState == SET_HOUR) {
      tempHour++;
      if (tempHour > 23) tempHour = 0;
    } else if (currentState == SET_MINUTE) {
      tempMinute++;
      if (tempMinute > 59) tempMinute = 0;
    }
  }

  // --- NORMAL VS SETTING OPERATIONS ---
  if (currentState == NORMAL) {
    int displayHour = currentHour24 % 12;
    if (displayHour == 0) displayHour = 12;
    display.showNumberDecEx(displayHour * 100 + now.minute(), 0b01000000, true);

    // --- 6:01 AM ROOSTER ALARM CHECK ---
    if (currentHour24 == 6 && currentMinute == 1) {
      if (now.day() != lastRoosterDay) {
        lastRoosterDay = now.day();
        musicPlayer.playFullFile("/rooster.mp3");
        announceTimeWithAnimation(currentHour24, currentMinute);
      }
    }

    // 15-Minute Automatic Check
    if ((currentMinute == 0 || currentMinute == 15 || currentMinute == 30 || currentMinute == 45)
        && currentMinute != lastMinuteChecked) {
      lastMinuteChecked = currentMinute;
      announceTimeWithAnimation(currentHour24, currentMinute);
    }

    if (currentMinute != 0 && currentMinute != 15 && currentMinute != 30 && currentMinute != 45) {
      lastMinuteChecked = -1;
```

```
    }

    // Manual Button Check
    if (digitalRead(SAY_TIME_BTN) == LOW) {
      if (millis() - lastButtonPress > debounceDelay) {
        lastButtonPress = millis();
        announceTimeWithAnimation(currentHour24, currentMinute);
      }
    }
  }
  else if (currentState == SET_HOUR) {
    display.showNumberDecEx(tempHour * 100 + 0, 0b01000000, true);
  }
  else if (currentState == SET_MINUTE) {
    display.showNumberDecEx(tempHour * 100 + tempMinute, 0b01000000, true);
  }

  delay(100);
}

void announceTimeWithAnimation(int hour24, int minute) {
  pulseLEDs();

  int displayHour = hour24 % 12;
  if (displayHour == 0) displayHour = 12;

  char audioFile[15];

  sprintf(audioFile, "/h%d.mp3", displayHour);
  musicPlayer.playFullFile(audioFile);

  if (minute <= 20) {
    sprintf(audioFile, "/m%d.mp3", minute);
    musicPlayer.playFullFile(audioFile);
  }
  else {
    int tens = (minute / 10) * 10;
    int units = minute % 10;

    sprintf(audioFile, "/m%d.mp3", tens);
    musicPlayer.playFullFile(audioFile);

    if (units > 0) {
      sprintf(audioFile, "/u%d.mp3", units);
      musicPlayer.playFullFile(audioFile);
    }
  }

  setGnomeIdleColor();
}

void setGnomeIdleColor() {
  for (int i = 0; i < strip.numPixels(); i++) {
    strip.setPixelColor(i, strip.Color(75, 0, 130));
  }
  strip.show();
}

void pulseLEDs() {
  for (int b = 50; b <= 255; b += 50) {
    for (int i = 0; i < strip.numPixels(); i++) {
      strip.setPixelColor(i, strip.Color(0, b, b));
    }
    strip.show();
```

```
    delay(50);
  }
}
```